

实验报告

一. 文档中要求回答的问题:

1. 之前没有过多人协同开发的经历
2. 可以灵活挑选提交内容 (保证每一次提交历史干净, 语义清晰); 提交前可以复核检查, 确认无误; 支持部分提交, 不需要将文件一次性改动提交;
3. git branch 仅查看本机本地分支, git branch-a 查看全部分支, 包含本地和远程服务器上的所有分支。

二. 实验步骤

1. 修改 main.c 文件, 修改 TODO 部分, 填入任意字符串。
2. 将修改加入暂存区, 提交 commit
3. feature 分支, 修改、提交, 制造冲突并解决冲突
4. 在 main 分支基础上创建并切换到 feature 分支
5. 修改 main.c 同一行 printf 语句, 保存文件, 提交
6. 切回 main 分支修改 main.c 同一行 printf 代码, 保存文件,
7. 提交 main 分支执行合并, 合并 feature 分支
8. Git 检测两个分支修改同一行代码, 自动合并失败, 文件出现冲突标记。
9. 打开 main.c, 删除冲突标记: <<<<<< HEAD、=====、>>>>>> feature, 保留 main 分支代码, 保存文件。完成冲突解决提交
10. 尝试推送远程仓库, 校园网访问 GitHub 返回 400 错误, 网络受限, 无法推送, 全部操作在本地仓库完成。

三. 阅读感悟

1.主要内容

commit message 规范: 这篇文章围绕 Git 的提交说明展开, 指出每次代码提交都必须填写 commit message, 介绍了单行与多行两种书写方式。文章重点介绍 Angular 的提交信息规范, 将提交信息划分为标题、正文、脚注三部分, 区分新增功能、修复 bug、文档修改等不同提交类型。遵循这套规范, 不仅便于团队查阅、梳理提交历史, 还可以借助工具自动生成项目版本更新日志 Change log, 提升项目版本维护的效率。

语义化版本: 本文介绍语义化版本控制, 指出普通版本号不利于依赖管理。该规范能清晰传递版本改动信息, 帮助开发者精准管理依赖, 规避版本兼容问题, 简化库的发布升级。

2.学习 Git 的原因:

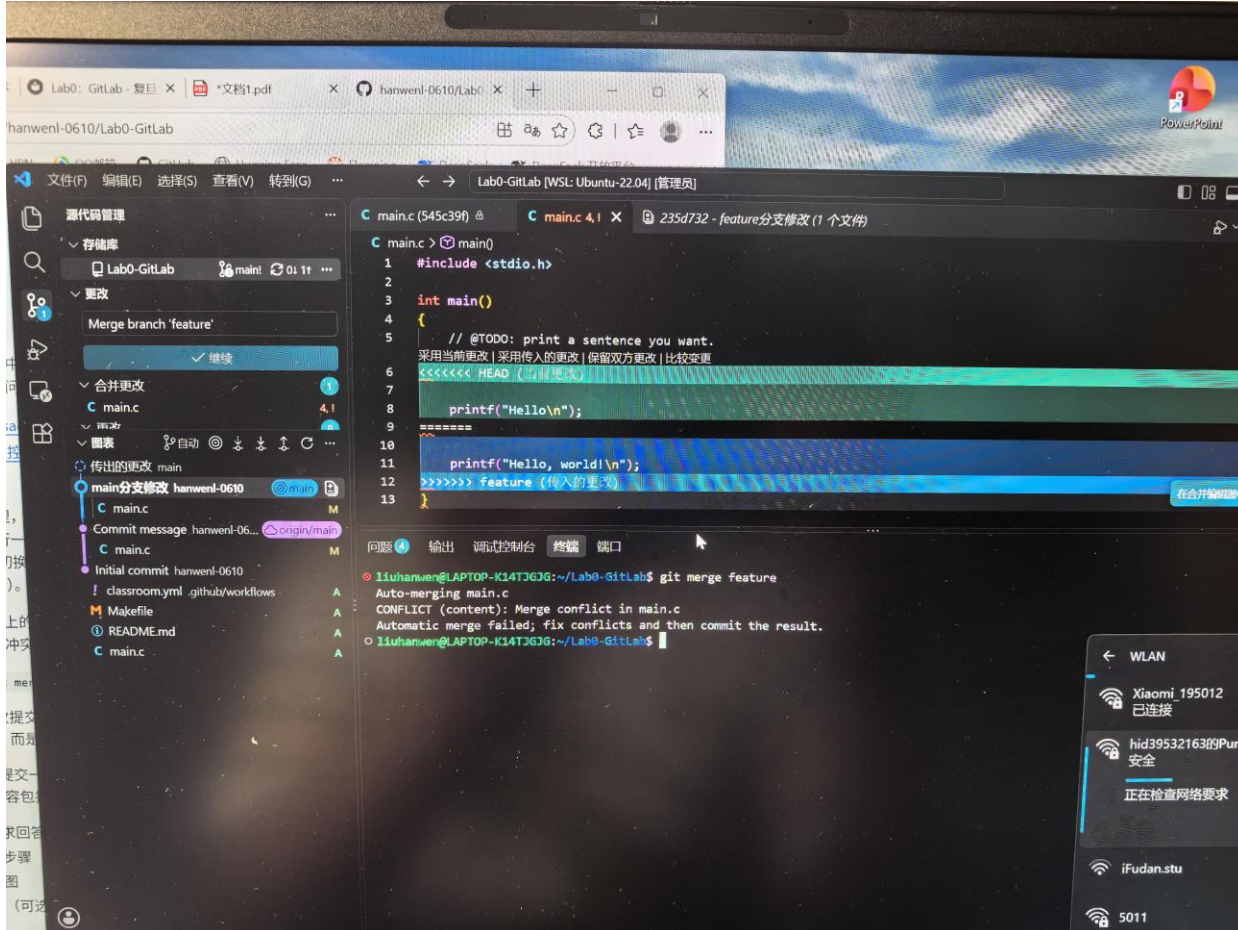
记录代码所有修改历史, 可以回溯、回退到过去任意版本。

方便多人协同开发, 多人可以并行修改代码, 合并各自改动。

备份代码, 防止文件误改、丢失。

支持创建分支, 新功能在分支上开发, 不破坏主代码。

方便项目代码分享、上交作业, 便于查看每次改动记录。



遇到了冲突

